

Appendix for "Hybrid Gaussians for Robust Open-Vocabulary 3D Segmentation with Multi-View Object Association and Boundary Refinement"

Anonymous ACCV 2026 Submission

Paper ID #341

1 3D Gaussian Rendering

Following standard 3DGS rendering [1, 6], the color or feature at a pixel is obtained by alpha compositing the contributions of Gaussians sorted along the corresponding viewing ray. For a generic per-Gaussian attribute r_n (e.g., RGB color or semantic feature), the compositing rule at pixel p in view u can be written as

$$\mathcal{R}(\mathcal{G}, u, p) = \sum_{n=1}^N \alpha_n r_n \prod_{m=1}^{n-1} (1 - \alpha_m), \quad (1)$$

where α_n denotes the pixel-wise opacity contribution of the n -th Gaussian at pixel p in view u , which is determined by its opacity δ_n together with its projected Gaussian response after depth sorting. Applying this compositing rule to all pixels yields the rendered image or feature map. In particular, object-conditioned rendering for object k is obtained by restricting the contributing Gaussians to the subset $\mathcal{G}_k \subset \mathcal{G}$ while using the same compositing rule.

2 More Implementation Details

2.1 2D Mask Segmentation and Multi-object Tracking

We adopt the 2D mask extraction and multi-object tracking pipeline of Segment-then-Splat [5] as the preprocessing stage of our work. Given the input image set $\mathcal{I} = \{I_u^{gt}\}_{u \in \mathcal{U}}$, this pipeline extracts multi-view object masks at multiple granularity levels, including large, middle, and small objects. It first applies SAM [3] with grid-based point prompting on the initial view to obtain object mask proposals at different scales, and then uses SAM2 [7] to propagate these masks across the remaining views and form object tracks.

Since direct SAM/SAM2 propagation may be affected by newly appearing objects, overlapping mask assignments, and interrupted tracks, this preprocessing stage augments training pipeline with additional heuristics for periodic object discovery, IoU-based conflict filtering, and recovery from tracking loss. We follow this preprocessing procedure to obtain the multi-view 2D object masks used in our subsequent object association and boundary learning stages.

2.2 Hyper-parameter Settings

We train each scene for 40,000 iterations using the Adam optimizer. The learning rates for Gaussian positions, SH features, opacity, scaling, rotation, and language features are set to 1.6×10^{-4} , 2.5×10^{-3} , 5×10^{-2} , 1×10^{-3} , 1×10^{-3} , and 1×10^{-3} , respectively. The position learning rate is exponentially decayed to 1.6×10^{-6} with a delay multiplier of 0.01.

For object-conditioned optimization, we randomly sample 3 objects per iteration. To suppress noisy partial supervision, we perform partial-mask filtering after 30,000 iterations. Specifically, we compute the IoU between the binarized rendered object mask and the corresponding 2D object mask, and only keep the object-conditioned supervision if the IoU is larger than 0.3. In all settings, background-to-object reassignment is disabled, while object-to-background reassignment is allowed.

2.3 Evaluation Metric Details

We evaluate the predicted binary object masks using three metrics: mean Intersection over Union (mIoU), mean pixel accuracy (mAcc), and mean Boundary Intersection over Union (mBIoU). Given a predicted mask $M_{u,k}^{pred}$ and its corresponding ground-truth mask $M_{u,k}^{gt}$ for object k in view u , the IoU is defined as

$$\text{IoU}_{u,k} = \frac{|M_{u,k}^{pred} \cap M_{u,k}^{gt}|}{|M_{u,k}^{pred} \cup M_{u,k}^{gt}|}. \quad (2)$$

The final mIoU is obtained by averaging the IoU scores over all query objects and views.

The pixel-wise accuracy is defined as

$$\text{Acc}_{u,k} = \frac{1}{|\Omega|} \sum_{(x,y) \in \Omega} \mathbb{I}[M_{u,k}^{pred}(x,y) = M_{u,k}^{gt}(x,y)], \quad (3)$$

where Ω denotes the set of all image pixels, and $\mathbb{I}[\cdot]$ is the indicator function. The final mAcc is obtained by averaging the accuracy scores over all query objects and views.

To further assess boundary alignment quality, we adopt Boundary IoU. Let $\partial M_{u,k}^{pred}$ and $\partial M_{u,k}^{gt}$ denote the boundaries of $M_{u,k}^{pred}$ and $M_{u,k}^{gt}$, respectively, and let $\mathcal{N}_d(\partial M_{u,k}^{pred})$ and $\mathcal{N}_d(\partial M_{u,k}^{gt})$ denote the corresponding boundary bands obtained by dilating the boundaries with radius d . The Boundary IoU is defined as

$$\text{BIoU}_{u,k} = \frac{|\mathcal{N}_d(\partial M_{u,k}^{pred}) \cap \mathcal{N}_d(\partial M_{u,k}^{gt})|}{|\mathcal{N}_d(\partial M_{u,k}^{pred}) \cup \mathcal{N}_d(\partial M_{u,k}^{gt})|}. \quad (4)$$

The final mBIoU is obtained by averaging the BIoU scores over all query objects and views.

Table 1: Ablation study of Observation Fusion (OF) and Semantic Contrastive Learning (SCL) on the 3D-OVS dataset in terms of mAcc (%).

OF	SCL	<i>bed</i>	<i>bench</i>	<i>room</i>	<i>sofa</i>	<i>lawn</i>	Avg.
-	-	96.64	96.33	95.98	98.61	99.68	97.45
✓	-	96.82	96.99	97.29	98.96	99.71	97.95
-	✓	96.83	97.01	97.14	98.68	99.69	97.87
✓	✓	96.83	97.03	97.30	99.16	99.75	98.01

Table 2: Ablation study of boundary reconstruction optimization (BRO) on the LERF dataset in terms of mBIoU (%).

BRO	<i>figurines</i>	<i>teatime</i>	<i>ramen</i>	<i>kitchen</i>	Avg.
-	52.37	59.96	62.23	51.53	56.52
✓	55.92	62.00	63.30	52.89	58.53

3 More Ablation Studies

We further evaluate the proposed Observation Fusion (OF) and Semantic Contrastive Learning (SCL). As shown in Table 1, introducing OF alone improves the average mAcc (%) on 3D-OVS from 97.45 to 97.95, while enabling SCL alone increases it to 97.87. When OF and SCL are jointly applied, the average mAcc further rises to 98.01, achieving the best overall performance. In particular, the largest gain is observed on the “room” scene, where the score improves from 95.98% to 97.30%. These results indicate that OF and SCL play complementary roles in improving object allocation reliability and semantic discriminability.

We finally evaluate the proposed boundary reconstruction optimization (BRO). As shown in Table 2, BRO improves the average mBIoU (%) from 56.52% to 58.53%. Consistent improvements are observed across all scenes, with gains of 3.55% on “figurines”, 2.04% on “teatime”, 1.07 on “ramen”, and 1.36 on “kitchen”. These results show that BRO effectively refines fine-grained boundary reconstruction and improves boundary alignment quality.

4 Runtime Analysis

Tables 3 and 4 compares per-iteration training time and inference time on a single NVIDIA RTX 4090 GPU. All methods are trained for 40,000 iterations.

On LERF, our method requires 75.99s per iteration on average, ranking second behind Seg-Splat. It is faster than G-Grouping, OpenGaussian, and LangSplat by 28.2%, 10.4%, and 15.3%, respectively. On 3D-OVS, our method again ranks second, with an average training time of 38.98s per iteration, improving over G-Grouping, OpenGaussian, and LangSplat by 35.2%, 15.7%, and 24.5%, respectively. Seg-Splat remains faster on both datasets, reflecting its lighter object-specific reconstruction pipeline.

Table 3: Per-iteration training time and inference time (s) on the LERF dataset [2].

Method	<i>figurines</i>		<i>teatime</i>		<i>ramen</i>		<i>kitchen</i>		Avg.	
	Train	Infer	Train	Infer	Train	Infer	Train	Infer	Train	Infer
G-Grouping [9]	152.48	270.82	105.20	238.05	80.38	245.09	85.33	257.56	105.85	252.88
OpenGaussian [8]	126.00	<u>178.33</u>	79.65	90.81	70.96	112.81	62.48	121.67	84.77	125.91
LangSplat [6]	130.18	171.22	81.73	114.37	78.18	133.30	68.73	<u>122.65</u>	89.71	<u>135.39</u>
Seg-Splat [5]	89.23	180.76	67.33	<u>101.34</u>	48.33	<u>132.02</u>	57.03	127.71	65.48	135.46
Ours	<u>103.80</u>	398.77	<u>70.33</u>	190.80	<u>68.28</u>	281.26	<u>61.53</u>	243.42	<u>75.99</u>	278.56

Table 4: Per-iteration training time and inference time (s) on the 3D-OVS dataset [4].

Method	<i>bed</i>		<i>bench</i>		<i>room</i>		<i>sofa</i>		<i>lawn</i>		Avg.	
	Train	Infer	Train	Infer	Train	Infer	Train	Infer	Train	Infer	Train	Infer
G-Grouping [9]	62.40	185.58	66.73	203.64	48.28	103.56	50.08	155.20	73.39	221.70	60.18	173.94
OpenGaussian [8]	45.73	<u>63.60</u>	48.98	69.82	44.68	59.09	41.36	<u>67.55</u>	50.48	74.13	46.25	<u>66.84</u>
LangSplat [6]	53.83	61.45	50.18	<u>70.52</u>	46.88	<u>59.61</u>	49.03	65.55	58.10	<u>74.95</u>	51.60	66.42
Seg-Splat [5]	26.27	69.33	31.25	84.37	20.34	59.75	28.02	69.61	37.41	75.84	28.66	71.78
Ours	<u>39.40</u>	155.90	<u>45.68</u>	105.74	<u>32.13</u>	72.39	<u>30.29</u>	73.35	<u>47.41</u>	137.22	<u>38.98</u>	108.92

For inference, our method requires 278.56s on average on LERF, which is slightly higher than G-Grouping and substantially higher than the remaining baselines. On 3D-OVS, however, it achieves an average inference time of 108.92s, which is 37.4% lower than G-Grouping, although still higher than OpenGaussian, LangSplat, and Seg-Splat. Overall, the proposed method maintains competitive training efficiency while introducing additional inference cost for object-conditioned rendering and semantic matching.

5 Memory Cost Analysis

Tables 5 and 6 compares the peak memory cost of different methods on LERF and 3D-OVS. On LERF, our method requires 3721.02 MB on average, substantially less than G-Grouping (10037.63 MB), while using more memory than OpenGaussian, LangSplat, and Seg-Splat. The memory cost ranges from 2690.00 MB on “ramen” to 4294.00 MB on “figurines”. On 3D-OVS, our method uses 2903.34 MB on average, again remaining below G-Grouping (3603.59 MB). Its peak memory ranges from 2026.00 MB on “room” to 3892.69 MB on “lawn”. The additional memory relative to OpenGaussian, LangSplat, and Seg-Splat is mainly associated with maintaining object assignments and rendering object-specific semantic representations during inference.

Overall, our method incurs a moderate memory overhead due to maintaining object assignments and rendering object-conditioned semantic representations. This additional cost supports more reliable object-level association and semantic discrimination, which leads to improved open-vocabulary segmentation performance.

Table 5: Peak memory cost (MB) on the LERF dataset.

Method	<i>figurines</i>	<i>teatime</i>	<i>ramen</i>	<i>kitchen</i>	Avg.
G-Grouping [9]	11234.53	9892.31	9041.69	9982.00	10037.63
OpenGaussian [8]	2308.00	2589.05	1356.14	2146.00	2099.80
LangSplat [6]	<u>3430.00</u>	2585.76	<u>1434.94</u>	2736.02	<u>2546.68</u>
Seg-Splat [5]	3463.18	3226.00	1652.00	<u>2637.00</u>	2744.55
Ours	4294.00	4116.00	2690.00	3784.07	3721.02

Table 6: Peak memory cost (MB) on the 3D-OVS dataset.

Method	<i>bed</i>	<i>bench</i>	<i>room</i>	<i>sofa</i>	<i>lawn</i>	Avg.
G-Grouping [9]	3150.73	3729.20	3012.00	3906.00	4220.00	3603.59
OpenGaussian [8]	1279.27	1658.87	939.18	1341.46	1782.01	1400.16
LangSplat [6]	<u>1280.00</u>	<u>1660.00</u>	<u>943.54</u>	<u>1390.62</u>	<u>1892.55</u>	<u>1433.34</u>
Seg-Splat [5]	1406.00	1884.00	1006.00	1544.00	2072.00	1582.40
Ours	2240.00	3748.00	2026.00	2610.00	3892.69	2903.34

6 Limitations

Our framework relies on object masks generated by SAM/SAM2 as pseudo-supervision. Although Multi-View Object Association consolidates noisy, incomplete, and view-dependent proposals into a consistent 3D representation, objects that are entirely missed at the 2D proposal stage cannot be reliably recovered.

In addition, our current framework does not explicitly model mesh-level or topology-aware geometric structure, which may limit its effectiveness on objects with highly complex topology or extremely thin structures. The proposed framework also introduces multiple object-conditioned optimization components, resulting in higher training complexity than simpler 3DGS-based segmentation pipelines. Future work will investigate more robust proposal-generation or proposal-refinement mechanisms, topology-aware geometric modeling for complex structures, and training-free or feed-forward alternatives to reduce scene-specific optimization overhead.

7 More Segmentation Results

We provide additional segmentation results in Figure 1 to Figure 4, which present the segmentation performance under different text queries. To further demonstrate the multi-view consistency of our method, we also include video demos with dynamic visualizations.



Fig. 1: Open-vocabulary segmentation results in the “bench” scene.

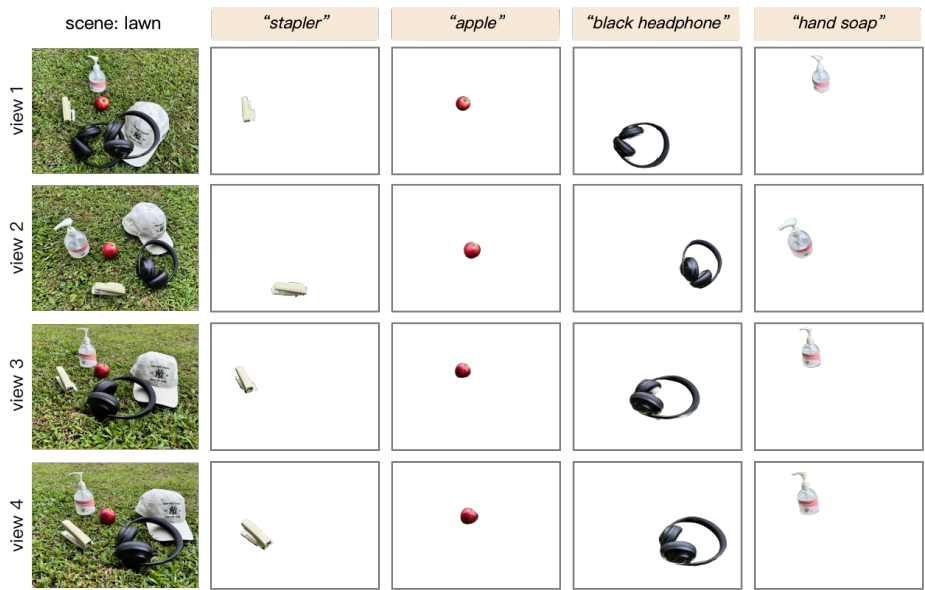


Fig. 2: Open-vocabulary segmentation results in the “lawn” scene.

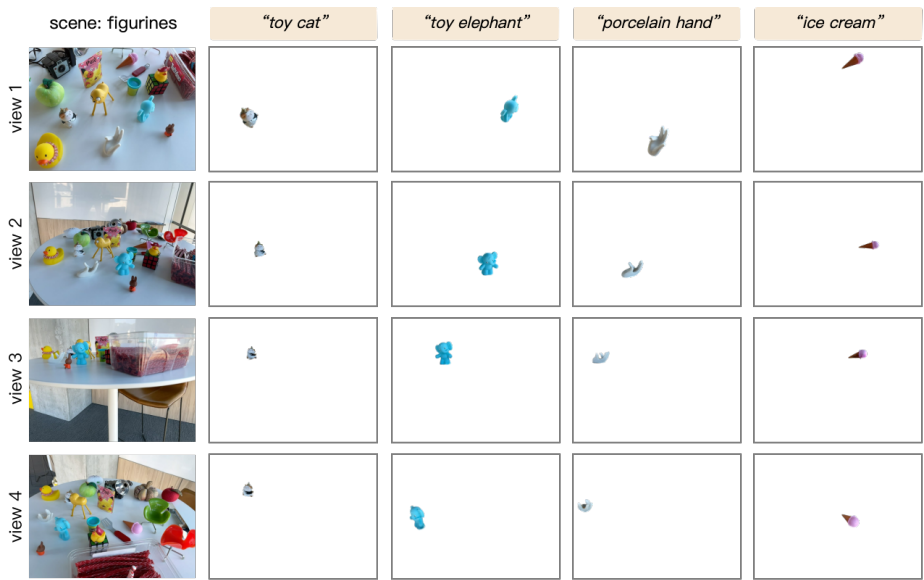


Fig. 3: Open-vocabulary segmentation results in the “figurines” scene.



Fig. 4: Open-vocabulary segmentation results in the “ramen” scene.

References

1. Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.* **42**(4), 139–1 (2023) 1

2. Kerr, J., Kim, C.M., Goldberg, K., Kanazawa, A., Tancik, M.: LERF: Language embedded radiance fields. In: *Proceedings of the IEEE/CVF international conference on computer vision*. pp. 19729–19739 (2023) 4

3. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., et al.: Segment anything. In: *Proceedings of the IEEE/CVF international conference on computer vision*. pp. 4015–4026 (2023) 1

4. Liu, K., Zhan, F., Zhang, J., Xu, M., Yu, Y., El Saddik, A., Theobalt, C., Xing, E., Lu, S.: Weakly supervised 3d open-vocabulary segmentation. *Advances in Neural Information Processing Systems* **36**, 53433–53456 (2023) 4

5. Lu, Y., Zhou, Y., Qiao, Y., Song, C., Liang, T., Ma, J., Wang, H., Yin, Y.: Segment then splat: Unified 3d open-vocabulary segmentation via gaussian splatting. *arXiv preprint arXiv:2503.22204* (2025) 1, 4, 5

6. Qin, M., Li, W., Zhou, J., Wang, H., Pfister, H.: LangSplat: 3d language gaussian splatting. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 20051–20060 (2024) 1, 4, 5

7. Ravi, N., Gabeur, V., Hu, Y.T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., et al.: SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714* (2024) 1

8. Wu, Y., Meng, J., Li, H., Wu, C., Shi, Y., Cheng, X., Zhao, C., Feng, H., Ding, E., Wang, J., et al.: Opengaussian: Towards point-level 3d gaussian-based open vocabulary understanding. *Advances in Neural Information Processing Systems* **37**, 19114–19138 (2024) 4, 5

9. Ye, M., Danelljan, M., Yu, F., Ke, L.: Gaussian grouping: Segment and edit anything in 3d scenes. In: *European conference on computer vision*. pp. 162–179. Springer (2024) 4, 5